

Sprawozdanie z laboratorium nr 1

Technologie Blockchain

Imię i nazwisko: Tomiczek Jakub Nr albumu: 67574

Data: 06.10.2026

CEL ĆWICZENIA

Celem ćwiczenia jest przygotowanie środowiska w Pythonie, implementacja szkieletu łańcucha bloków (klasa Blockchain) oraz doświadczalne zbadanie, w jaki sposób powiązanie bloków skrótami SHA-256 pozwala wykrywać manipulacje danymi i gdzie leżą granice tego mechanizmu.

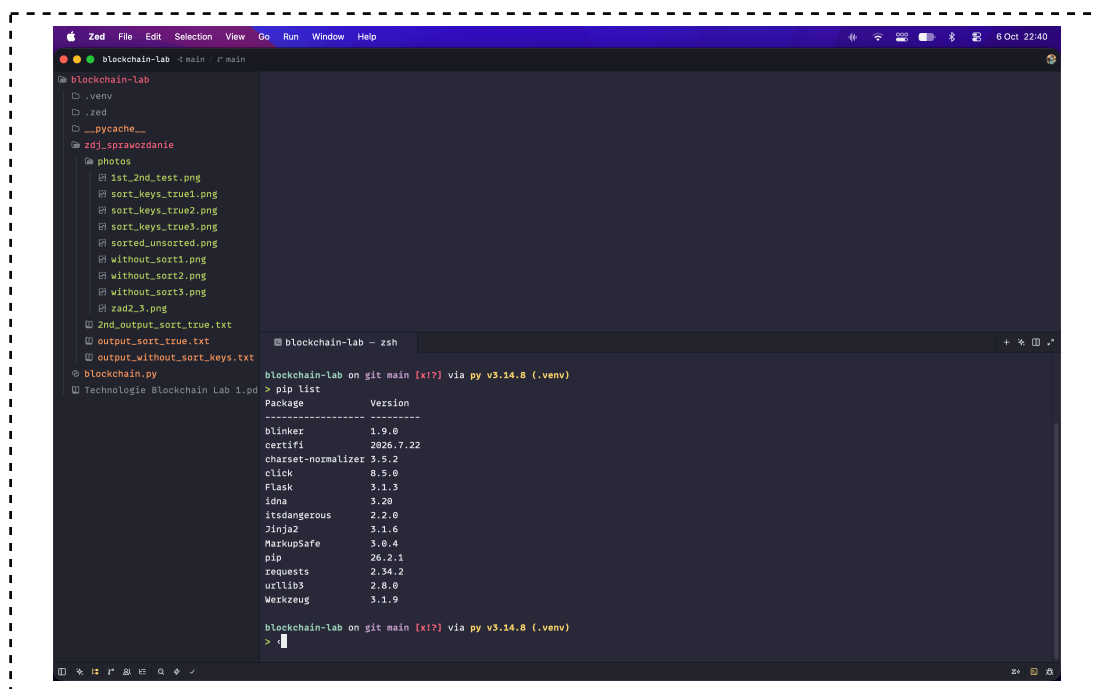
REALIZACJA ZADAŃ

Przygotowanie środowiska

Zainstalowano Python oraz edytor Zed i utworzono projekt blockchain-lab z własnym środowiskiem wirtualnym (venv). Następnie zainstalowano wymagane pakiety:

```
pip install flask requests
```

Wynik polecenia `pip list`:



```
blockchain-lab on git main [x1?] via py v3.14.8 (.venv)
> pip list
Package      Version
-----
blinker      1.9.0
certifi      2026.7.22
charset-normalizer 3.5.2
click        8.5.0
Flask        3.1.3
idna         3.20
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.4
pip          26.2.1
requests     2.34.2
urllib3      2.0.0
Werkzeug     3.1.9

blockchain-lab on git main [x1?] via py v3.14.8 (.venv)
>
```

Rysunek 1: Zainstalowane pakiety w środowisku wirtualnym

Implementacja szkieletu blockchaina

Klasa Blockchain

Zaimplementowano klasę Blockchain. Blok jest słownikiem o polach `index`, `timestamp`, `transactions`, `nonce` i `previous_hash`. Skrót bloku jest liczony jako SHA-256 z jego serializacji JSON (z posortowanymi kluczami), a pole `previous_hash` każdego nowego bloku zawiera skrót bloku poprzedniego. Blok genezy

ma `previous_hash` złożony z 64 zer. Do klasy dodano również metody `is_chain_valid()` (punkt 2.3) oraz `repair_as_attacker()` symulującą atak.

```
import hashlib
import json
from time import time

class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis = {
            "index": 1,
            "timestamp": 0,
            "transactions": [],
            "nonce": 0,
            "previous_hash": "0" * 64,
        }
        self.chain.append(genesis)

    @staticmethod
    def hash(block):
        # sort_keys=True gwarantuje deterministyczną kolejność pól
        block_string = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

    @property
    def last_block(self):
        return self.chain[-1]

    def add_block(self, transactions):
        block = {
            "index": len(self.chain) + 1,
            "timestamp": time(),
            "transactions": transactions,
            "nonce": 0,
            "previous_hash": self.hash(self.last_block),
        }
        self.chain.append(block)
        return block

    def is_chain_valid(self, chain):
        for i in range(1, len(chain)):
            current_block = chain[i]
            previous_block = chain[i - 1]
            if current_block["previous_hash"] != self.hash(previous_block):
                return False, i + 1

        return True, None

    def repair_as_attacker(self, start):
```

```
for i in range(start + 1, len(self.chain)):
    self.chain[i]["previous_hash"] = self.hash(self.chain[i - 1])
```

Dodanie bloków i wyświetlenie łańcucha

Do łańcucha dodano trzy bloki z przykładowymi transakcjami i wyświetlono zawartość każdego bloku wraz z jego skrótem:

```
if __name__ == "__main__":
    bc = Blockchain()
    bc.add_block(["Alice -> Bob: 5"])
    bc.add_block(["Bob -> Carol: 2"])
    bc.add_block(["Bob -> Carol: 3"])

    for block in bc.chain:
        print(json.dumps(block, indent=2, ensure_ascii=False))
        print("hash:", Blockchain.hash(block))
        print("-" * 70)
        print()
```

Wynik działania programu (wartości timestamp i skróty zależą od chwili uruchomienia):

```
{
  "index": 1,
  "timestamp": 0,
  "transactions": [],
  "nonce": 0,
  "previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172
-----

{
  "index": 2,
  "timestamp": 1791393944.690767,
  "transactions": [
    "Alice -> Bob: 5"
  ],
  "nonce": 0,
  "previous_hash": "147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172"
}
hash: 4c419b769832647d29a36de5db0d014516e48591a88e4dcdf1100f1fb69434d1
-----

{
  "index": 3,
  "timestamp": 1791393944.69079,
  "transactions": [
    "Bob -> Carol: 2"
  ],
  "nonce": 0,
  "previous_hash": "4c419b769832647d29a36de5db0d014516e48591a88e4dcdf1100f1fb69434d1"
}
hash: c38bcd90fa2a00c81d3701f4a03345f3ac727e32ffb5fd59348d908a346213
```

```

-----
{
  "index": 4,
  "timestamp": 1791393944.690805,
  "transactions": [
    "Bob -> Carol: 3"
  ],
  "nonce": 0,
  "previous_hash": "c38bcd90fa2a00c81d3701f4a03345f3ac727e32ffb5fd59348d908a346213"
}
hash: 58be1698cdb21e12ec4c733b2ebece9afa777f5b9f1edc08cbfd0913be98f539
-----

```

Sprawdzenie powiązań: pole `previous_hash` bloku nr 2 (147aed30...) jest równe skrótowi bloku nr 1, `previous_hash` bloku nr 3 (be12c0d3...) skrótowi bloku nr 2, a `previous_hash` bloku nr 4 (2d31a956...) skrótowi bloku nr 3. Każdy blok wskazuje zatem na właściwego poprzednika. Blok genezy wskazuje na ciąg zer, ponieważ nie ma poprzednika.

```

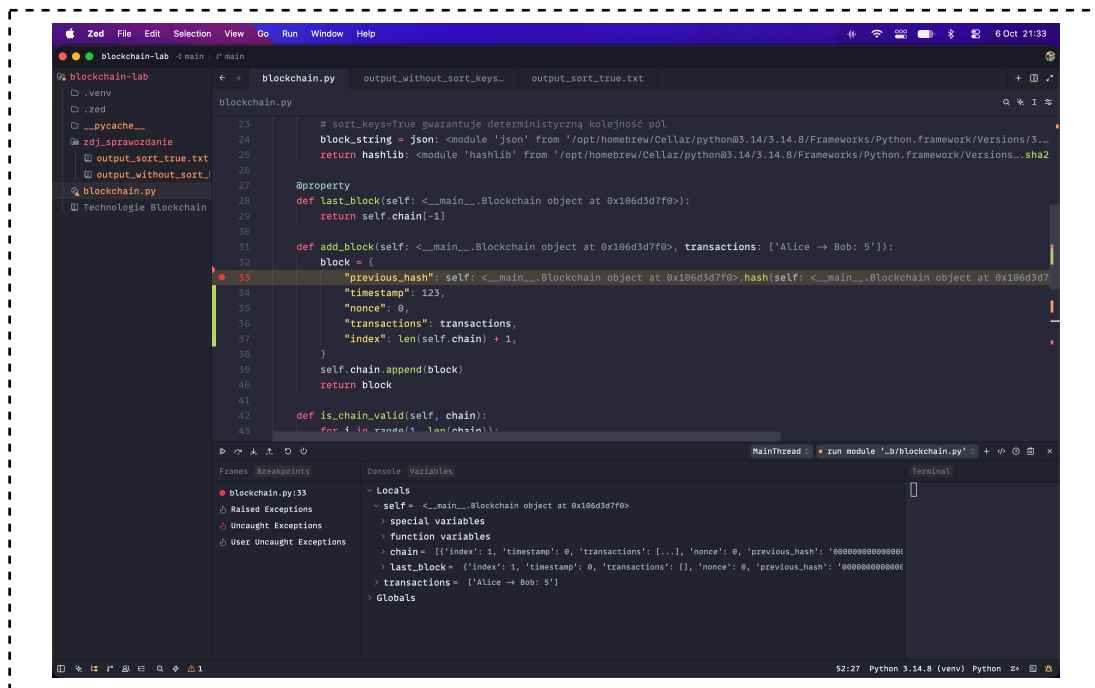
Zed File Edit Selection View Go Run Window Help
blockchain-lab - zsh
blockchain-lab on git main via py v3.14.8 (.venv)
> python blockchain.py
{
  "index": 1,
  "timestamp": 0,
  "transactions": [],
  "nonce": 0,
  "previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 147aed30e21a13f96f72aab4f74ca20fc513b08e18ad6caa74db678786640172
-----
{
  "index": 2,
  "timestamp": 1791393944.690767,
  "transactions": [
    "Alice -> Bob: 5"
  ],
  "nonce": 0,
  "previous_hash": "147aed30e21a13f96f72aab4f74ca20fc513b08e18ad6caa74db678786640172"
}
hash: 4c419b769832647d29a36de5dbd0d14516e48591a88e4dcd1180f1fb69434d1
-----
{
  "index": 3,
  "timestamp": 1791393944.69079,
  "transactions": [
    "Bob -> Carol: 2"
  ],
  "nonce": 0,
  "previous_hash": "4c419b769832647d29a36de5dbd0d14516e48591a88e4dcd1180f1fb69434d1"
}
hash: c38bcd90fa2a00c81d3701f4a03345f3ac727e32ffb5fd59348d908a346213
-----
{
  "index": 4,
  "timestamp": 1791393944.690805,
  "transactions": [
    "Bob -> Carol: 3"
  ],
  "nonce": 0,
  "previous_hash": "c38bcd90fa2a00c81d3701f4a03345f3ac727e32ffb5fd59348d908a346213"
}
hash: 58be1698cdb21e12ec4c733b2ebece9afa777f5b9f1edc08cbfd0913be98f539
-----
15:32 Python 3.14.8 (.venv) Python 3.14.8

```

Rysunek 2: Zawartość łańcucha wraz ze skrótami bloków

Debugowanie metody `add_block()`

Ustawiono punkt przerwania w metodzie `add_block()` i prześledzono w debuggerze tworzenie nowego bloku.



Rysunek 3: Okno Variables podczas wykonywania add_block()

W oknie Variables widoczne są m.in. argument transactions, wartość `len(self.chain) + 1` użyta jako index nowego bloku oraz skrót ostatniego bloku obliczony do pola `previous_hash`.

Wpływ parametru `sort_keys=True`

W metodzie `hash()` usunięto parametr `sort_keys=True` i porównano skróty tego samego bloku zapisanego z dwiema różnymi kolejnościami pól:

```
import hashlib, json

a = {"index": 2, "timestamp": 1.0, "transactions": ["Alice -> Bob: 5"],
     "nonce": 0, "previous_hash": "abc"}
b = {"previous_hash": "abc", "nonce": 0, "transactions": ["Alice -> Bob: 5"],
     "timestamp": 1.0, "index": 2}

def h(x, s):
    return hashlib.sha256(json.dumps(x, sort_keys=s).encode()).hexdigest()

print("bez sort_keys, kolejnosc A:", h(a, False))
print("bez sort_keys, kolejnosc B:", h(b, False))
print("z sort_keys,    kolejnosc A:", h(a, True))
print("z sort_keys,    kolejnosc B:", h(b, True))
```

Wynik:

```
bez sort_keys, kolejnosc A:
59d9ef61d066235cee8127daee4bde75d0992f5a7d0edd0377ad1d62494576c
bez sort_keys, kolejnosc B:
899e85d40e8ca79e1898b2d7ef7b0a09dfa60bfce4c0bd0492854fa17d394201
z sort_keys,    kolejnosc A:
72614c90b45196efb1b2b5713a5950bbc8c31d999c30574c3e798cfd313b6f24
z sort_keys,    kolejnosc B:
72614c90b45196efb1b2b5713a5950bbc8c31d999c30574c3e798cfd313b6f24
```

Wyjaśnienie. Bez `sort_keys=True` funkcja `json.dumps` zapisuje pola w kolejności, w jakiej występują w słowniku. Dwa bloki o identycznej zawartości, lecz innej kolejności pól, dają więc różne ciągi znaków, a SHA-256 (dzięki efektowi lawinowemu) zwraca dla nich całkowicie różne skróty. Poprawny blok zostałby wtedy uznany za zmodyfikowany, np. po odebraniu go od innego węzła, który zapisał pola w innej kolejności. Z `sort_keys=True` serializacja jest kanoniczna i ten sam blok zawsze daje ten sam skrót.

Weryfikacja integralności

Metoda `is_chain_valid()`

Metoda `is_chain_valid()` dla każdego bloku (od drugiego) porównuje zapisany w nim `previous_hash` z aktualnie obliczonym skrótem bloku poprzedniego. Zwraca (`True`, `None`) dla poprawnego łańcucha albo (`False`, `numer_bloku`) z numerem pierwszego niepoprawnego bloku (numeracja zgodna z polem `index`, od 1). Kod metody przedstawiono w punkcie 2.2.1.

Scenariusze ataku

Przeprowadzono kolejno trzy scenariusze:

```
bc.chain[1]["transactions"][0] = "Alice -> Bob: 500"
is_valid, index = bc.is_chain_valid(bc.chain)
print("1) Chain valid:", is_valid, "index:", index)

bc.repair_as_attacker(1)
is_valid, index = bc.is_chain_valid(bc.chain)
print("2) Chain valid:", is_valid, "index:", index)

bc.chain[-1]["transactions"][0] = "Bob -> Carol: 3000"
is_valid, index = bc.is_chain_valid(bc.chain)
print("3) Chain valid:", is_valid, "index:", index)
```

Wynik:

```
1) Chain valid: False index: 3
2) Chain valid: True index: None
3) Chain valid: True index: None
```

```
blockchain.py
56 if __name__ == "__main__":
57     bc = Blockchain()
58     bc.add_block(["Alice -> Bob: 500"])
59     bc.add_block(["Bob -> Carol: 200"])
60     bc.add_block(["Bob -> Carol: 3000"])
61
62     for block in bc.chain:
63         print(json.dumps(block, indent=2, ensure_ascii=False))
64         print("hash:", Blockchain.hash(block))
65         print("-" * 70)
66         print()
67
68     bc.chain[1]["transactions"][0] = "Alice -> Bob: 500"
69     is_valid, index = bc.is_chain_valid(bc.chain)
70     print("1) Chain valid:", is_valid, "index:", index)
71
72     bc.repair_as_attacker(1)
73     is_valid, index = bc.is_chain_valid(bc.chain)
74     print("2) Chain valid:", is_valid, "index:", index)
75
76     bc.chain[-1]["transactions"][0] = "Bob -> Carol: 3000"
77     is_valid, index = bc.is_chain_valid(bc.chain)
78     print("3) Chain valid:", is_valid, "index:", index)
79
```

```
blockchain-lab -- zsh
1) Chain valid: False index: 3
2) Chain valid: True index: None
3) Chain valid: True index: None

blockchain-lab on git main [x17] via py v3.14.8 (.venv)
>
```

Rysunek 4: Wyniki weryfikacji łańcucha po kolejnych modyfikacjach

Scenariusz 1: zmiana kwoty w bloku nr 2. Metoda wykryła manipulację i zwróciła (False, 3). Zmiana treści bloku nr 2 zmieniła jego skrót, więc zapisany w bloku nr 3 previous_hash przestał się z nim zgadzać. Pierwszą niezgodność widać w bloku nr 3, a nie w zmodyfikowanym bloku nr 2, ponieważ blok przechowuje skrót swojego poprzednika, a nie własny.

Scenariusz 2: „naprawa” łańcucha przez atakującego. Po przeliczeniu i nadpisaniu pól previous_hash we wszystkich blokach następujących po zmodyfikowanym metoda zwraca (True, None), czyli ataku już nie wykrywa. Sprawdza ona wyłącznie spójność powiązań skrótów, a wszystkie dane potrzebne do ich odtworzenia atakujący ma w samym łańcuchu. W zaimplementowanym szkielecie nic nie utrudnia przeliczenia skrótów: pole nonce jest stałe (brak proof-of-work), nie ma podpisów cyfrowych ani zewnętrznej kopii referencyjnej.

Scenariusz 3: zmiana ostatniego bloku. Metoda również zwraca (True, None). Skrót ostatniego bloku nie jest zapisany w żadnym następnym bloku, więc nie ma z czym go porównać, a weryfikacja sprawdza tylko bloki mające następnika. Wykrycie takiej zmiany wymaga zewnętrznego odniesienia, np. zapamiętanego wcześniej skrótu ostatniego bloku lub porównania z kopiami łańcucha przechowywanymi przez inne węzły.

ODPOWIEDZI NA PYTANIA KONTROLNE

1. Jakie właściwości funkcji skrótu decydują o bezpieczeństwie łańcucha bloków?

Kluczowe są:

- (a) *jednokierunkowość*, czyli brak możliwości odtworzenia danych ze skrótu;
- (b) *odporność na kolizję*, czyli prawie zerowa szansa znalezienia innego bloku o tym samym skrót;
- (c) *efekt lawinowy*, czyli zmiana jednego bitu danych całkowicie zmienia skrót, więc każda modyfikacja jest widoczna;
- (d) *deterministyczność* te same dane zawsze dają ten sam skrót, stała długość wyniku i szybkie obliczanie, co umożliwia łatwą weryfikację.

2. Dlaczego przy obliczaniu skrótu bloku stosuje się serializację z posortowanymi kluczami?

Skrót zależy od dokładnej postaci bajtowej danych. Słownik o tej samej zawartości, lecz innej kolejności

pól, zostałyby bez sortowania zapisany inaczej i dałby inny skrót (co pokazano w punkcie 2.2.4). Posortowane klucze zapewniają, że wszystkie węzły obliczają identyczny skrót tego samego bloku.

3. Czym różni się blok genezy od pozostałych bloków i dlaczego musi być identyczny we wszystkich węzłach?

Blok genezy jest pierwszy, więc nie ma poprzednika: jego `previous_hash` to stała wartość (same zera), a nie skrót innego bloku. Jest zapisany na stałe (także `timestamp` równy 0, a nie bieżący czas), a nie tworzony z transakcji w trakcie działania sieci. Jego skrót wchodzi do `previous_hash` bloku nr 2, a przez to pośrednio do wszystkich kolejnych skrótów. Jeżeli węzły miałyby różne bloki genezy, każdy z nich miałby inny łańcuch, a bloki jednego węzła nie przeszłyby weryfikacji u drugiego.

4. Dlaczego sam łańcuch skrótów nie chroni przed atakującym, który przeliczy wszystkie kolejne bloki?

Skróty można obliczyć na podstawie samych danych łańcucha, a ich obliczenie jest tanie. Atakujący zmienia blok, po czym przelicza `previous_hash` w kolejnych blokach i otrzymuje łańcuch wewnętrznie spójny (scenariusz 2). Dodatkowo zmiana ostatniego bloku nie jest wykrywana wcale (scenariusz 3). Ochronę zapewniają dopiero dodatkowe mechanizmy: `proof-of-work` (przeliczenie wymagałoby ponownego wykonania kosztownych obliczeń dla każdego bloku), podpisy cyfrowe oraz rozproszenie kopii łańcucha między wiele węzłów, które odrzuca sfalszowaną wersję.

WNIOSKI

1. Powiązanie bloków skrótami SHA-256 sprawia, że zmiana dowolnego bloku unieważnia pole `previous_hash` bloku następnego.
2. Serializacja (`sort_keys=True`) jest konieczna, aby ten sam blok zawsze dawał ten sam skrót.
3. Sam łańcuch skrótów nie chroni przed atakującym, który przelicza kolejne bloki, a zmiana ostatniego bloku w ogóle nie jest wykrywana.
4. Realna odporność na manipulację wymaga `proof-of-work`, podpisów cyfrowych i rozproszenia kopii łańcucha między węzły.

KOD ŹRÓDŁOWY

Pełny kod programu znajduje się w pliku `blockchain.py` w projekcie `blockchain-lab` (lub: link do repozytorium <https://github.com/EddieCarbon/blockchain-lab.git>).